

```

1 from sklearn.ensemble import AdaBoostClassifier # подключим модуль
2 from sklearn import datasets # наборы данных
3 from sklearn.model_selection import train_test_split # разделение
4 from sklearn import metrics # метрики
5 iris=datasets.load_iris() # загрузим набор данных ирисов
6 X=iris.data # входы (признаки)
7 y=iris.target # указания учителя
8 # разделим данные на обучающие и тестовые
9 X_train, X_test, y_train, y_test=train_test_split(X, y, test_size=0.3)
10 # создадим Adaboost классификатор, по умолчанию- на основе деревьев
11 abc=AdaBoostClassifier(n_estimators=50, learning_rate=1)
12 # обучим его
13 model=abc.fit(X_train, y_train)
14 # проверим на тесте
15 y_pred=model.predict(X_test)
16 # посчитаем метрики
17 print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

```

Accuracy: 0.9777777777777777

```

1 abc.feature_importances_

```

array([0.06179818, 0.04940355, 0.33201822, 0.55678004])

```

1 from sklearn.svm import SVC
2 # создаем базовый классификатор
3 svc=SVC(probability=True, kernel='rbf')
4 # создаем Adaboost классификатор из них
5 abc=AdaBoostClassifier(n_estimators=50, learning_rate=1)
6 # обучаем
7 model=abc.fit(X_train, y_train)
8 # тестируем
9 y_pred=model.predict(X_test)
10 # считаем метрики
11 print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

```

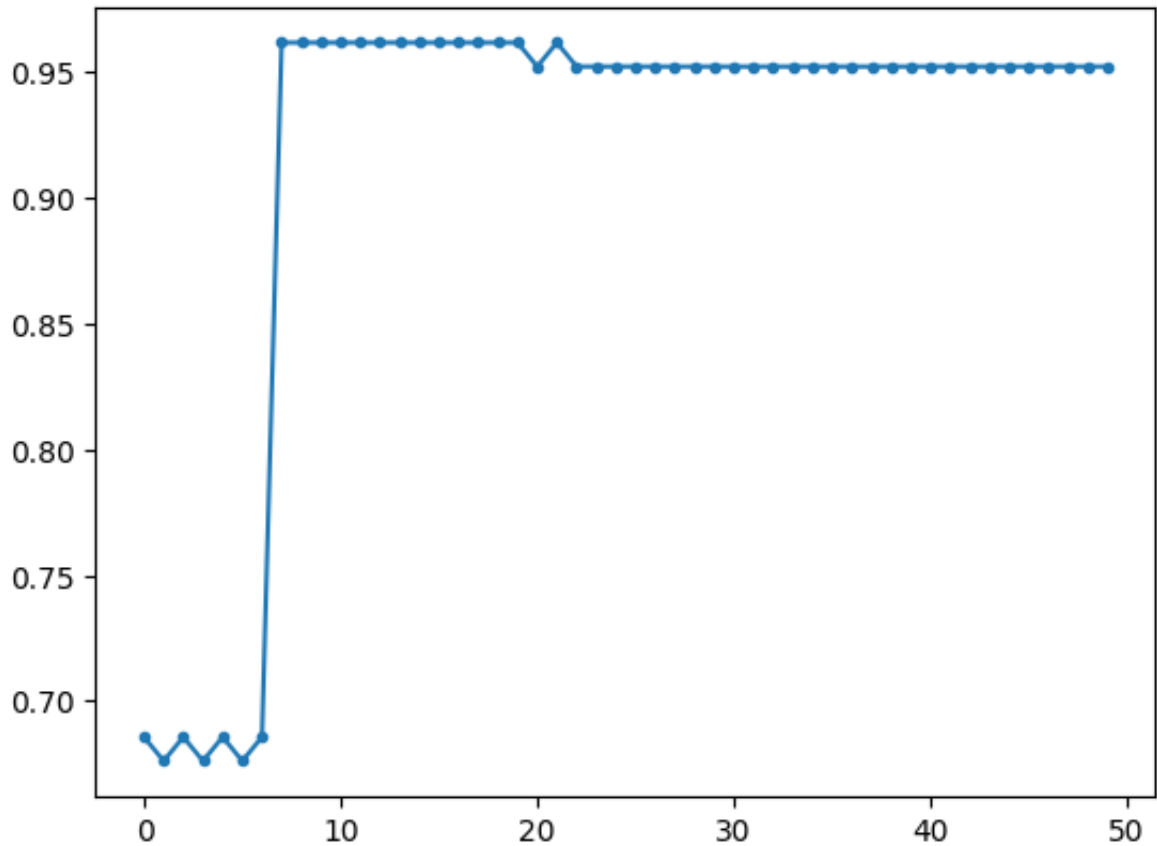
Accuracy: 0.9777777777777777

```

1 # посмотрим как изменялась ошибка при обучении
2 from matplotlib import pyplot as plt
3 learning_rate=0.1
4 abc=AdaBoostClassifier(n_estimators=50, learning_rate=learning_rate)
5 model=abc.fit(X_train, y_train)
6 score=[ s for s in abc.staged_score(X_train, y_train)]
7 plt.plot(score,marker='.')
8 #plt.yscale('log')

```

[<matplotlib.lines.Line2D at 0x7abb63332c00>]



```

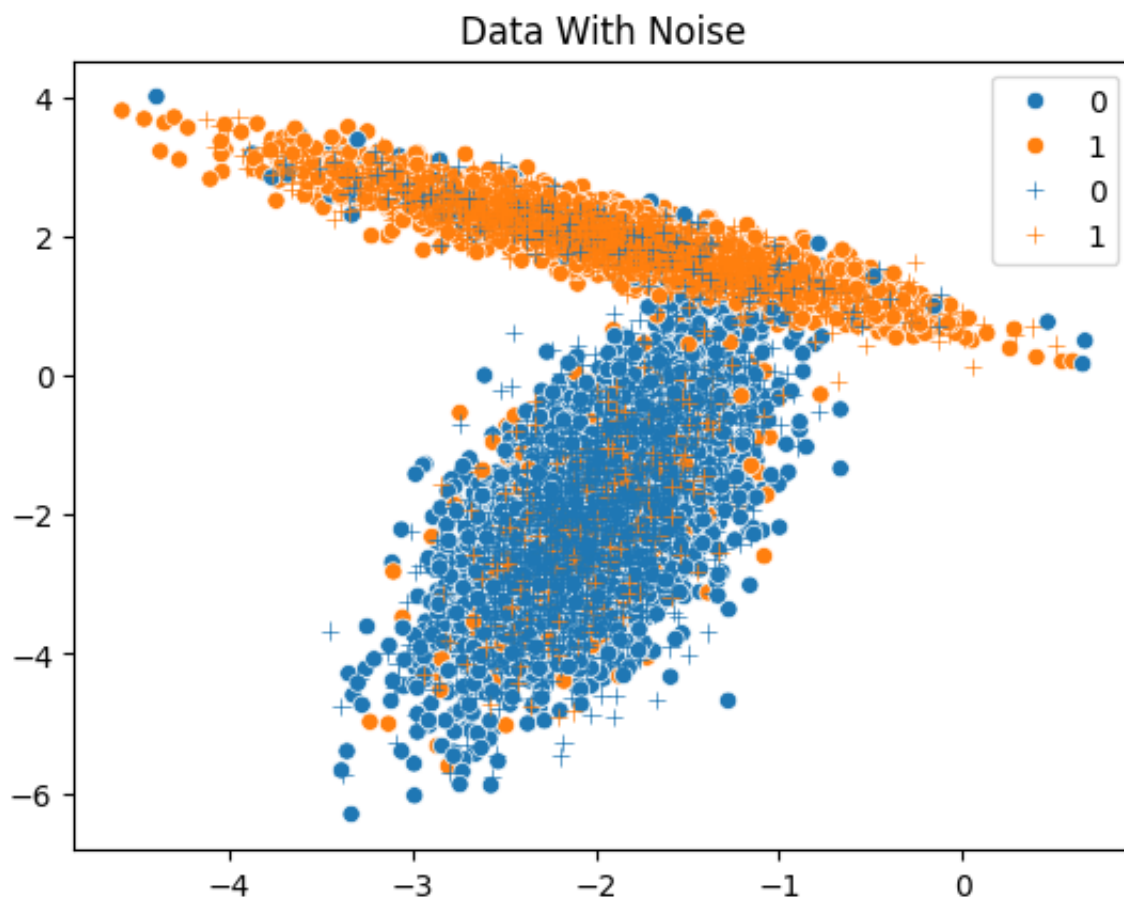
1 # функция для рисования разделяющей поверхности на плоскости
2 # Div_plate(clf,flip_level) принимает объект-классификатор и уровень
3 # создает двумерные данные, обучает классификатор, рисует разделяющую
4 def Div_plate(clf, flip_level):
5     from sklearn.datasets import make_classification # для генерации
6     from sklearn.model_selection import train_test_split # для разделения
7     import numpy as np
8     import seaborn as sns # для простого отображения
9     import matplotlib.pyplot as plt #
10    # создаем данные
11    X,y = make_classification(n_samples=10000, # число примеров
12                             n_features=2, # число признаков (атрибутов)
13                             n_informative=2, # из них информативных
14                             n_redundant=0, # из них не информативных
15                             n_repeated=0, # из них повторяющихся
16                             n_classes=2, # число классов
17                             n_clusters_per_class=1, # число кластеров

```

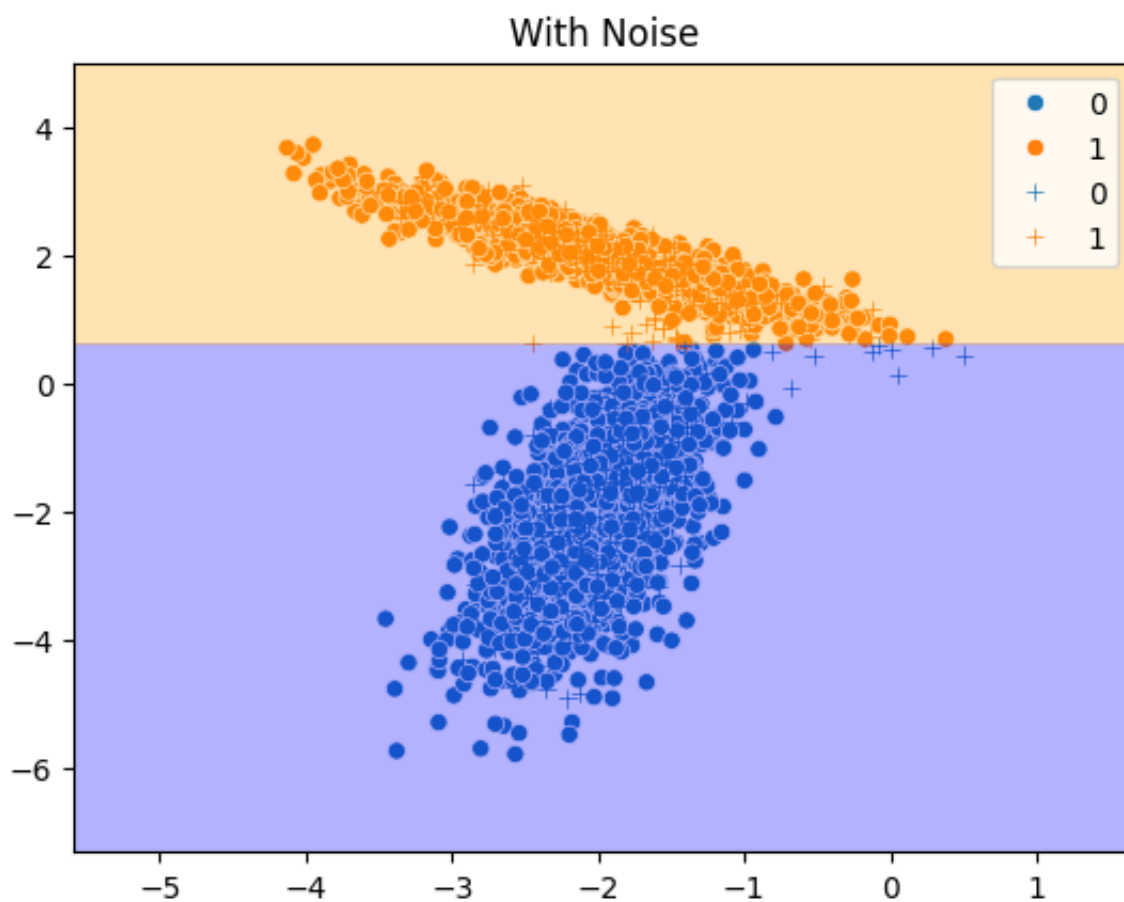
```

18         class_sep=2, # влияет на расстояние между
19         flip_y=flip_level, # доля ошибок (шума)
20         weights=[0.5,0.5], # пропорции числа дан
21         random_state=17) #
22 # разделяем на обучающие и тестовые, случайно
23 X_train, X_test, y_train, y_test = train_test_split(
24     X, y, test_size=0.33, random_state=42)
25 # рисуем данные
26 plt.subplots();#
27 ax1=plt.gca();#
28 sns.scatterplot(x=X_train[:,0],y=X_train[:,1],hue=y_train,ax=ax1)
29 sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test,ax=ax1,marker='x')
30 ax1.set_title("Data With Noise");#
31 plt.show();#
32 # Создаем классификатор на основе случайного леса. Изменяйте параметры
33 clf.fit(X_train, y_train) # обучаем
34 y_pred = clf.predict(X_test) # проверяем на тестовых данных
35 score=clf.score(X_test, y_test) # считаем среднюю точность
36 print('Score=',score)
37 ind=y_test==y_pred; # индексы совпадений результата классификации
38 plt.subplots();
39 ax2=plt.gca();
40 # рисуем "правильно" распознанные примеры
41 sns.scatterplot(x=X_test[ind,0],y=X_test[ind,1],hue=y_test[ind],marker='x')
42 # рисуем "неправильно" распознанные примеры
43 sns.scatterplot(x=X_test[~ind,0],y=X_test[~ind,1],hue=y_pred[~ind],marker='x')
44 ax2.set_title("With Noise");
45 # считаем и рисуем разделяющую поверхность.
46 plot_step=0.01
47 x_min, x_max = X[:, 0].min()-1, X[:, 0].max()+1 # немного изменим границы
48 y_min, y_max = X[:, 1].min()-1, X[:, 1].max()+1 # немного изменим границы
49 # считаем прямоугольную сетку возможных значений этих атрибутов
50 xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #
51                     np.arange(y_min, y_max, plot_step)) #
52 # считаем выход классификатора для всех примеров сетки
53 # не забыв что массивы данных нужно привести к требуемому размеру
54 Z=clf.predict(np.c_[xx.ravel(), yy.ravel()])
55 Z=Z.reshape(xx.shape) # и преобразуем обратно в исходному размеру
56 # рисуем разделяющую поверхность
57 cs=plt.contourf(xx, yy, Z, levels=1, colors=['blue','orange'],alpha=0.5)
58 plt.show();
59 from sklearn.ensemble import AdaBoostClassifier
60 from sklearn.linear_model import LogisticRegression
61 # создаем Adaboost классификатор из них
62 #clf=LogisticRegression()
63 abc=AdaBoostClassifier(# base_estimator=clf,
64 n_estimators=100,
65 learning_rate=1) # дерево по умолчанию
66 flip_level=0.2 # уровень шума
67 Div_plate(abc,flip_level) # рисуем разделяющие поверхности
68 Score= 0.8896969696969697

```



Score= 0.89



```

1 import numpy #
2 import xgboost # подключим библиотеку XGboost
3 from sklearn.metrics import accuracy_score #
4 from sklearn.model_selection import train_test_split #
5 # загрузим данные для классификации
6 URL='https://drive.google.com/uc?id=1soReu6FBB_4LxvuNnV_ULUW-WGlna
7 dataset=numpy.loadtxt(URL, delimiter=",")
8 # в них последний столбец это класс, остальные-признаки
9 X=dataset[:,0:8] # помним, что в срезе последний элемент не входит
10 Y=dataset[:,8] #
11 # разделяем данные на обучающие и тестовые
12 seed=7
13 test_size=0.33
14 X_train, X_test, y_train, y_test=train_test_split(X, Y, test_size=
15 # обучаем градиентный бустинг деревьев (по умолчанию)
16 model=xgboost.XGBClassifier() # создаем объект
17 model.fit(X_train, y_train) # обучаем его
18 # вычисляем выход на тесте
19 predictions=model.predict(X_test) #
20 # считаем точность
21 accuracy=accuracy_score(y_test, predictions) #
22 print("Accuracy: %.2f%%" % (accuracy*100.0))

```

Accuracy: 72.44%

```
1 # установка библиотеки
2 !pip install catboost
```

Collecting catboost

```
Downloading catboost-1.2.8-cp312-cp312-manylinux2014_x86_64.whl.metadata
Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: numpy<3.0, >=1.16.0 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from catboost)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from catboost)
Downloading catboost-1.2.8-cp312-cp312-manylinux2014_x86_64.whl (99.2 MB)
99.2/99.2 MB 5.5 MB/s eta 0s
```

Installing collected packages: catboost

Successfully installed catboost-1.2.8

```

1 from catboost import Pool, CatBoostClassifier # подключаем инструм
2 # данные для обучения—одна категориальная и две вещественные пере
3 train_data=[["summer", 1924, 44],
4             ["summer", 1932, 37],
5             ["winter", 1980, 37],
6             ["summer", 2012, 204]]
7 # данные для теста
8 eval_data=[["winter", 1996, 197],
9            ["winter", 1968, 37],
10           ["summer", 2002, 77],
11           ["summer", 1948, 59]]
12 cat_features=[0] # номера категориальных переменных
13 train_label=["France", "USA", "USA", "UK"] # указания учителя для
14 eval_label=["USA", "France", "USA", "UK"] # укзания учителя для те
15
16 # создаем обучающий набор данных
17 train_dataset=Pool(data=train_data, # сами данные
18                   label=train_label, # указания учителя
19                   cat_features=cat_features) # индексы категори
20 # аналогично для тестовых данных
21 eval_dataset=Pool(data=eval_data, #
22                  label=eval_label, #
23                  cat_features=cat_features) #
24 # создаем модель CatBoostClassifier
25 model=CatBoostClassifier(iterations=10, # число моделей в ансамбле
26                          learning_rate=1, # шаг обучения
27                          depth=2, # глубина дерева
28                          loss_function='MultiClass') # вид функции потерь
29 # обучаем ансамбль
30 model.fit(train_dataset)
31 # считаем выходы после обучения на тестовых данных
32 preds_class=model.predict(eval_dataset)
33 # считаем вероятности классов
34 preds_proba=model.predict_proba(eval_dataset)
35 preds_class

```

```

0:      learn: 0.9417331      total: 47.6ms      remaining: 428ms
1:      learn: 0.8421839      total: 48ms       remaining: 192ms
2:      learn: 0.6597822      total: 48.1ms    remaining: 112ms
3:      learn: 0.6028493      total: 48.2ms    remaining: 72.2ms
4:      learn: 0.4900112      total: 48.2ms    remaining: 48.2ms
5:      learn: 0.4076408      total: 48.3ms    remaining: 32.2ms
6:      learn: 0.3458205      total: 48.3ms    remaining: 20.7ms
7:      learn: 0.2982687      total: 48.4ms    remaining: 12.1ms
8:      learn: 0.2608927      total: 48.4ms    remaining: 5.38ms
9:      learn: 0.2309514      total: 48.5ms    remaining: 0us
array([[ 'USA'],
       [ 'USA'],
       [ 'UK'],
       [ 'USA']], dtype=object)

```

```

1 from catboost import Pool, cv # подключаем модуль
2 # данные для обучения и кроссвалидации
3 cv_data=[["France", 1924, 44],
4           ["USA", 1932, 37],
5           ["Switzerland", 1928, 25],
6           ["Norway", 1952, 30],
7           ["Japan", 1972, 35],
8           ["Mexico", 1968, 112]]
9 # указания учителя
10 labels=[1, 1, 0, 0, 0, 1]
11 cat_features=[0] # категориальные переменные
12 # создаем обучающий набор
13 cv_dataset=Pool(data=cv_data, # данные
14                 label=labels, # указания учителя
15                 cat_features=cat_features) # категориальные переменные
16 # параметры обучения
17 params={"iterations": 100, # максимальное число моделей в ансамбле
18         "depth": 2, # глубина дерева
19         "loss_function": "Logloss", # вид функции ошибки
20         "verbose": False} # отображение процесса текстом
21 # обучение и кроссвалидация
22 scores=cv(cv_dataset, # данные на которых обучаем
23           params, # параметры обучения
24           fold_count=3, # число разбиений данных для кроссвалидации
25           plot=False) # отображение интерактивных графиков

```

Training on fold [0/3]

```

bestTest = 0.6806661468
bestIteration = 1

```

Training on fold [1/3]

```

bestTest = 0.7062352628
bestIteration = 0

```

Training on fold [2/3]

```

bestTest = 0.693188484
bestIteration = 0

```


1 scores

	iterations	test- Logloss- mean	test- Logloss- std	train- Logloss- mean	train- Logloss- std
0	0	0.701886	0.007533	0.686716	7.850462e-17
1	1	0.693624	0.012898	0.661645	1.110223e-16
2	2	0.702647	0.006738	0.637718	0.000000e+00
3	3	0.716449	0.014089	0.625042	8.796724e-03
4	4	0.710532	0.006287	0.606618	8.448647e-03
...	...	...	...	...	...
95	95	1.102985	0.195439	0.099079	7.536046e-03
96	96	1.109511	0.194170	0.097707	7.360085e-03
97	97	1.115997	0.192900	0.096368	7.189676e-03
98	98	1.122444	0.191630	0.095060	7.024605e-03
99	99	1.128853	0.190359	0.093782	6.864665e-03

Далее:

[Создать код с переменной scores](#)

[New interactive sheet](#)

```

1 import numpy as np
2 from catboost import CatBoostRegressor, Pool # перепечоп
3 import pandas as pd
4 import seaborn as sns
5 from matplotlib import pyplot as plt
6 # from sklearn.datasets import load_boston # уже исключен из набор
7 from sklearn.model_selection import train_test_split
8 from sklearn.metrics import mean_squared_error
9 from sklearn.metrics import r2_score
10 from sklearn.inspection import permutation_importance # для оценки
11 from sklearn.datasets import fetch_california_housing
12 housing=fetch_california_housing()
13 X,y=housing.data, housing.target
14 X_train, X_test, y_train, y_test=train_test_split(X, y, test_size=
15 train_dataset=Pool(X_train, y_train) # создаем пул обучающих данных
16 test_dataset=Pool(X_test, y_test) # создаем пул тестовых данных
17 model=CatBoostRegressor(loss_function='RMSE') # создаем модель для

```

```

1 # задаем "сетку" параметров, варианты, которые нужно проверить
2 grid={'iterations': [100, 150, 200],
3       'learning_rate': [0.03, 0.1],
4       'depth': [2, 4, 6, 8],
5       'l2_leaf_reg': [0.2, 0.5, 1, 3]}

```

6 model.grid_search(grid, train_dataset) # и обучаем модели со всеми

190:	learn: 0.5281821	test: 0.5358348	best: 0.5358348	(190)
191:	learn: 0.5277805	test: 0.5357040	best: 0.5357040	(191)
192:	learn: 0.5274275	test: 0.5354847	best: 0.5354847	(192)
193:	learn: 0.5271850	test: 0.5352837	best: 0.5352837	(193)
194:	learn: 0.5266965	test: 0.5347753	best: 0.5347753	(194)
195:	learn: 0.5263330	test: 0.5345125	best: 0.5345125	(195)
196:	learn: 0.5259718	test: 0.5342126	best: 0.5342126	(196)
197:	learn: 0.5257481	test: 0.5340537	best: 0.5340537	(197)
198:	learn: 0.5250513	test: 0.5334966	best: 0.5334966	(198)
199:	learn: 0.5247517	test: 0.5333758	best: 0.5333758	(199)

bestTest = 0.5333757525

bestIteration = 199

68:	loss: 0.5333758	best: 0.4744028 (67)	total: 39.8s	rema
0:	learn: 2.1540531	test: 2.1680790	best: 2.1680790	(0)
1:	learn: 1.9720383	test: 1.9843087	best: 1.9843087	(1)
2:	learn: 1.8090344	test: 1.8200476	best: 1.8200476	(2)
3:	learn: 1.6623716	test: 1.6714102	best: 1.6714102	(3)
4:	learn: 1.5329052	test: 1.5405518	best: 1.5405518	(4)
5:	learn: 1.4169144	test: 1.4229644	best: 1.4229644	(5)
6:	learn: 1.3151493	test: 1.3211625	best: 1.3211625	(6)
7:	learn: 1.2234525	test: 1.2301748	best: 1.2301748	(7)
8:	learn: 1.1455502	test: 1.1527334	best: 1.1527334	(8)
9:	learn: 1.0785762	test: 1.0849399	best: 1.0849399	(9)
10:	learn: 1.0155367	test: 1.0214757	best: 1.0214757	(10)
11:	learn: 0.9610359	test: 0.9670516	best: 0.9670516	(11)
12:	learn: 0.9109235	test: 0.9165378	best: 0.9165378	(12)
13:	learn: 0.8681969	test: 0.8736792	best: 0.8736792	(13)
14:	learn: 0.8329843	test: 0.8379016	best: 0.8379016	(14)
15:	learn: 0.8023853	test: 0.8073173	best: 0.8073173	(15)
16:	learn: 0.7758648	test: 0.7803521	best: 0.7803521	(16)
17:	learn: 0.7506250	test: 0.7545518	best: 0.7545518	(17)
18:	learn: 0.7293644	test: 0.7333198	best: 0.7333198	(18)
19:	learn: 0.7104149	test: 0.7142229	best: 0.7142229	(19)
20:	learn: 0.6936570	test: 0.6974473	best: 0.6974473	(20)
21:	learn: 0.6763795	test: 0.6798817	best: 0.6798817	(21)
22:	learn: 0.6643341	test: 0.6675146	best: 0.6675146	(22)
23:	learn: 0.6550673	test: 0.6582303	best: 0.6582303	(23)
24:	learn: 0.6443217	test: 0.6477079	best: 0.6477079	(24)
25:	learn: 0.6333222	test: 0.6368191	best: 0.6368191	(25)
26:	learn: 0.6243987	test: 0.6276165	best: 0.6276165	(26)
27:	learn: 0.6167087	test: 0.6190114	best: 0.6190114	(27)
28:	learn: 0.6095852	test: 0.6118645	best: 0.6118645	(28)
29:	learn: 0.6027593	test: 0.6048964	best: 0.6048964	(29)
30:	learn: 0.5974556	test: 0.6001778	best: 0.6001778	(30)
31:	learn: 0.5914778	test: 0.5943611	best: 0.5943611	(31)
32:	learn: 0.5867064	test: 0.5898878	best: 0.5898878	(32)
33:	learn: 0.5830371	test: 0.5865298	best: 0.5865298	(33)
34:	learn: 0.5802350	test: 0.5841650	best: 0.5841650	(34)
35:	learn: 0.5758200	test: 0.5796213	best: 0.5796213	(35)
36:	learn: 0.5724871	test: 0.5762154	best: 0.5762154	(36)
37:	learn: 0.5682719	test: 0.5716489	best: 0.5716489	(37)
38:	learn: 0.5656490	test: 0.5691970	best: 0.5691970	(38)
39:	learn: 0.5628842	test: 0.5663435	best: 0.5663435	(39)

```

40:      learn: 0.5602191      test: 0.5638735 best: 0.5638735 (40)
41:      learn: 0.5581614      test: 0.5620356 best: 0.5620356 (41)
42:      learn: 0.5551962      test: 0.5591116 best: 0.5591116 (42)
43:      learn: 0.5522362      test: 0.5570772 best: 0.5570772 (43)

```

```

1 pred=model.predict(X_test) # рассчитываем выход (лучшая модель из
2 rmse=(np.sqrt(mean_squared_error(y_test, pred))) # ошибка rmse
3 r2=r2_score(y_test, pred) # точность r2
4 print('Testing performance')
5 print('RMSE: {:.2f}'.format(rmse))
6 print('R2: {:.2f}'.format(r2))

```

```

Testing performance
RMSE: 0.46
R2: 0.85

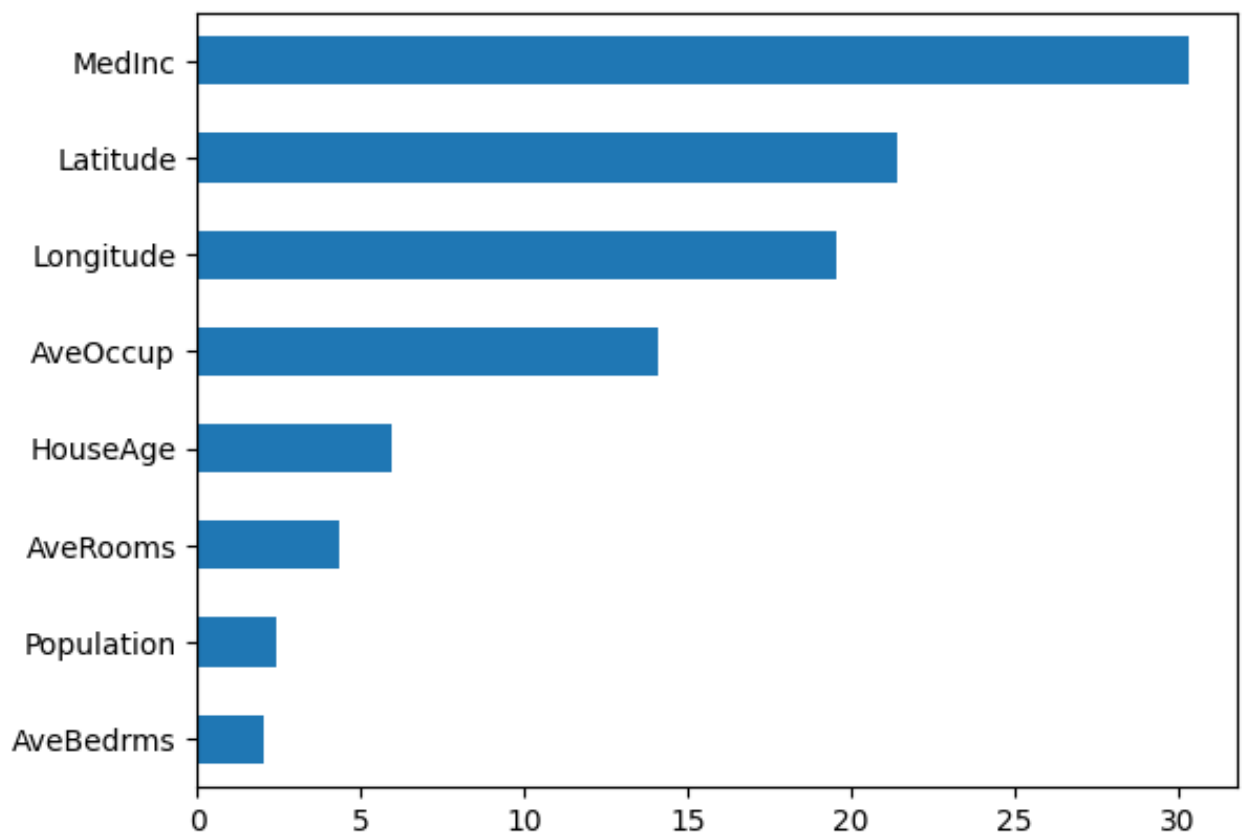
```

```

1 sorted_feature_importance=model.feature_importances_.argsort() # с
2 # и построим график
3 (pd.Series(model.feature_importances_, index=housing.feature_names
4     .sort_values()
5     .plot(kind='barh'))

```

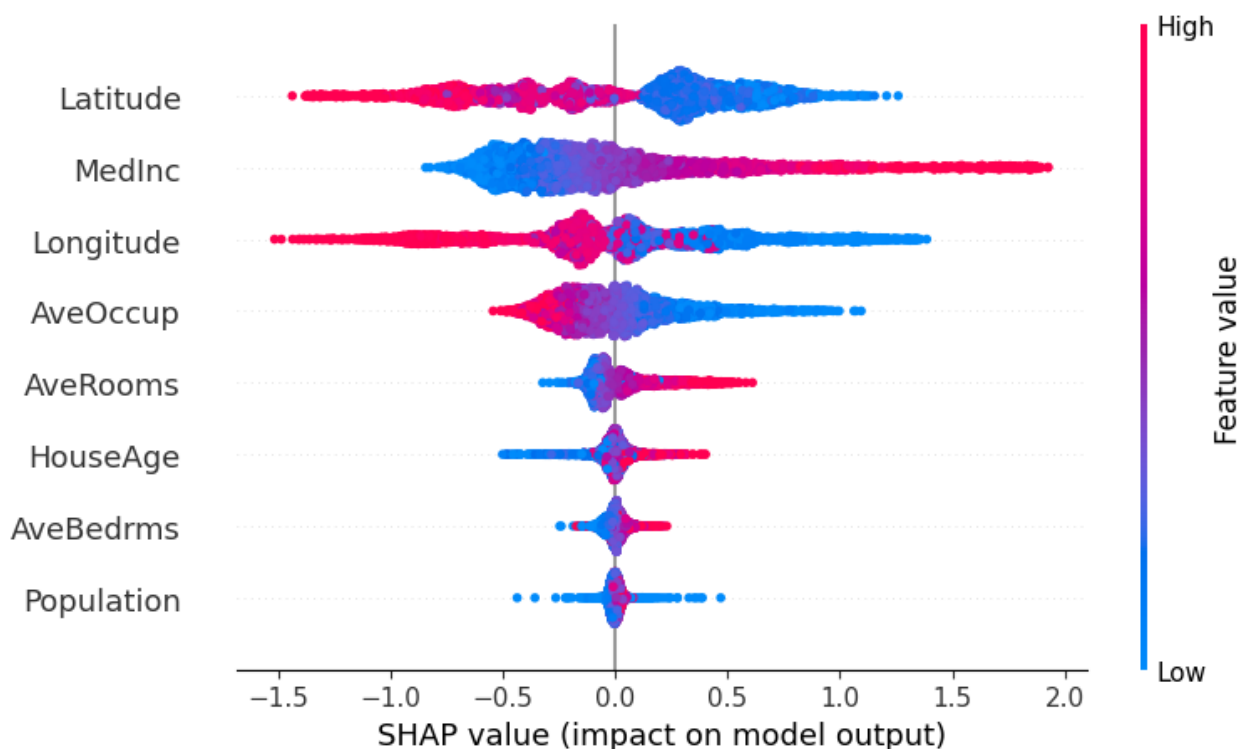
<Axes: >



```
1 !pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
```

```
1 import shap # подключим библиотеку  
2 explainer=shap.TreeExplainer(model) # "объяснитель" для ансамблей  
3 shap_values=explainer.shap_values(X_test) # рассчитываем вклад при  
4 shap.summary_plot(shap_values, X_test, feature_names=housing.feature
```



```

1 from sklearn.linear_model import Ridge # модель, которую будем испо
2 model_ridge=Ridge() # создаем
3 model_ridge.fit(X_train,y_train) # обучаем
4 model_ridge_train=model_ridge.predict(X_train) # считаем выходы для
5 model_ridge_test=model_ridge.predict(X_test) # и тестовых данных
6 train_dataset=Pool(X_train, y_train, baseline=model_ridge_train) #
7 test_dataset=Pool(X_test, y_test, baseline=model_ridge_test) # пул
8 model=CatBoostRegressor(loss_function='RMSE') # создаем ансамбль
9 model.fit(train_dataset,eval_set=test_dataset) # обучаем
10 pred=model.predict(test_dataset) # считаем выходы
11 r2_ridge=r2_score(y_test, model_ridge_test) # точность базовой моде
12 r2=r2_score(y_test, pred) # точность ансамбля
13 print('Testing performance')
14 print('R2 Ridge: {:.2f}'.format(r2_ridge))
15 print('R2 Catboost: {:.2f}'.format(r2))

```

Learning rate set to 0.07828

0:	learn: 0.7115167	test: 0.7213193	best: 0.7213193 (0)
1:	learn: 0.7019438	test: 0.7110709	best: 0.7110709 (1)
2:	learn: 0.6932241	test: 0.7018858	best: 0.7018858 (2)
3:	learn: 0.6851460	test: 0.6930552	best: 0.6930552 (3)
4:	learn: 0.6775051	test: 0.6856935	best: 0.6856935 (4)
5:	learn: 0.6701618	test: 0.6777533	best: 0.6777533 (5)
6:	learn: 0.6638747	test: 0.6713760	best: 0.6713760 (6)
7:	learn: 0.6580430	test: 0.6651202	best: 0.6651202 (7)
8:	learn: 0.6528298	test: 0.6601031	best: 0.6601031 (8)
9:	learn: 0.6478665	test: 0.6547171	best: 0.6547171 (9)
10:	learn: 0.6430612	test: 0.6503921	best: 0.6503921 (10)
11:	learn: 0.6380951	test: 0.6453410	best: 0.6453410 (11)
12:	learn: 0.6334311	test: 0.6410162	best: 0.6410162 (12)
13:	learn: 0.6298080	test: 0.6373494	best: 0.6373494 (13)
14:	learn: 0.6268852	test: 0.6344402	best: 0.6344402 (14)
15:	learn: 0.6238051	test: 0.6310407	best: 0.6310407 (15)
16:	learn: 0.6207473	test: 0.6279988	best: 0.6279988 (16)
17:	learn: 0.6181899	test: 0.6258051	best: 0.6258051 (17)
18:	learn: 0.6140544	test: 0.6215838	best: 0.6215838 (18)
19:	learn: 0.6108146	test: 0.6181600	best: 0.6181600 (19)
20:	learn: 0.6085218	test: 0.6160503	best: 0.6160503 (20)
21:	learn: 0.6061688	test: 0.6135383	best: 0.6135383 (21)
22:	learn: 0.6032476	test: 0.6105477	best: 0.6105477 (22)
23:	learn: 0.6009852	test: 0.6084567	best: 0.6084567 (23)
24:	learn: 0.5987781	test: 0.6063390	best: 0.6063390 (24)
25:	learn: 0.5970140	test: 0.6046621	best: 0.6046621 (25)
26:	learn: 0.5951076	test: 0.6029080	best: 0.6029080 (26)
27:	learn: 0.5936599	test: 0.6012110	best: 0.6012110 (27)
28:	learn: 0.5919269	test: 0.5994657	best: 0.5994657 (28)
29:	learn: 0.5906034	test: 0.5982567	best: 0.5982567 (29)
30:	learn: 0.5877970	test: 0.5953209	best: 0.5953209 (30)
31:	learn: 0.5858703	test: 0.5935988	best: 0.5935988 (31)
32:	learn: 0.5844534	test: 0.5922213	best: 0.5922213 (32)
33:	learn: 0.5823882	test: 0.5901819	best: 0.5901819 (33)
34:	learn: 0.5806254	test: 0.5884871	best: 0.5884871 (34)
35:	learn: 0.5796621	test: 0.5879295	best: 0.5879295 (35)
36:	learn: 0.5785129	test: 0.5868968	best: 0.5868968 (36)
37:	learn: 0.5765923	test: 0.5851206	best: 0.5851206 (37)

38:	learn: 0.5741881	test: 0.5827889	best: 0.5827889 (38)
39:	learn: 0.5727162	test: 0.5812004	best: 0.5812004 (39)
40:	learn: 0.5716385	test: 0.5803095	best: 0.5803095 (40)
41:	learn: 0.5701730	test: 0.5791672	best: 0.5791672 (41)
42:	learn: 0.5683232	test: 0.5776373	best: 0.5776373 (42)
43:	learn: 0.5671210	test: 0.5766201	best: 0.5766201 (43)
44:	learn: 0.5657513	test: 0.5754436	best: 0.5754436 (44)
45:	learn: 0.5644206	test: 0.5742927	best: 0.5742927 (45)
46:	learn: 0.5631507	test: 0.5734978	best: 0.5734978 (46)
47:	learn: 0.5613671	test: 0.5718816	best: 0.5718816 (47)
48:	learn: 0.5604896	test: 0.5711777	best: 0.5711777 (48)
49:	learn: 0.5596483	test: 0.5704140	best: 0.5704140 (49)
50:	learn: 0.5582413	test: 0.5691388	best: 0.5691388 (50)
51:	learn: 0.5571468	test: 0.5682596	best: 0.5682596 (51)
52:	learn: 0.5554054	test: 0.5665584	best: 0.5665584 (52)
53:	learn: 0.5544587	test: 0.5657710	best: 0.5657710 (53)
54:	learn: 0.5537968	test: 0.5652038	best: 0.5652038 (54)
55:	learn: 0.5525995	test: 0.5645254	best: 0.5645254 (55)
56:	learn: 0.5514102	test: 0.5635603	best: 0.5635603 (56)
--			